ASSIGNMENT

INTELLIGENT DISASTER EARLY WARNING AND EVACUATION MANAGEMENT SYSTEM

Assam Flood Early Warning System

Team Member	Register Number
K R Gayana	192471039
S Keerthana	192471047
N Ramya	192511403
A Saruhaasini	192511400
S Nithiyasri	192572082
M Hashetha	192521162

1.1Problem Statement

Design, plan, and build a full-stack, containerized “Intelligent Disaster Early Warning and Evacuation Management System” that integrates IoT sensors, weather and seismic monitoring, AI/ML, GIS, and cloud technologies to improve disaster preparedness and emergency response. Using Agile project management (Scrum), decompose the problem into a prioritized product backlog and organize development across multiple sprints with defined sprint goals and a visible Scrum board. Carry out detailed requirements analysis from citizen, disaster-management authority, emergency response agency, and administrator perspectives; model these using use-case diagrams and user stories; and prioritize features using a recognized technique such as MoSCoW. Design a scalable architecture covering real-time sensor-data collection, AI/ML-based disaster-risk prediction, location-specific multi-channel alerts, GIS-based affected-area mapping and evacuation-route recommendations, shelter occupancy and resource monitoring, citizen emergency reporting, and an authority dashboard. Build a working full-stack implementation with frontend, backend API, database, and GIS/AI services; containerize the components using Docker/Docker Compose; and manage the codebase collaboratively on GitHub using feature branches, pull requests, and code reviews.

Assessment Rubrics

Assignment Title: Intelligent Disaster Early Warning and Evacuation Management System

1.2 Problem Formulation

Assam is vulnerable to recurrent monsoon flooding because of heavy rainfall, river overflow, erosion, drainage congestion, and settlements located close to the Brahmaputra and its tributaries. Flood information may be distributed across sensors, river-gauge stations, weather services, field officers, and citizen reports. When these sources are not integrated, warnings may be delayed, insufficiently localised, or difficult for citizens and authorities to act upon. The assignment addresses this coordination and decision-support problem through an intelligent, full-stack, containerized system.

The problem is to design and build an Assam Flood Early Warning and Evacuation Management System that collects flood-related data, estimates risk for geographic zones, issues location-specific multi-channel alerts, maps affected areas, recommends evacuation routes, monitors shelters and resources, and supports communication

between citizens, emergency responders, disaster-management authorities, and administrators.

Formally, for each monitoring zone z at time t, the system receives water level, rainfall, weather, GIS, historical, vulnerability, and citizen-report data. It must calculate a risk value R(z,t), classify it as Normal, Watch, Warning, or Severe Warning, and recommend an appropriate response. The system should maximise warning usefulness and minimise false alarms, delay, data loss, and unsafe route recommendations while preserving privacy and maintaining human authority over final evacuation decisions.

Input	Processing	Output
Water level, rainfall, weather	Validation and time-series processing	Clean monitoring data
GIS, elevation, roads, shelters	Spatial analysis and routing	Hazard map and evacuation routes
Historical flood observations	Risk modelling and threshold rules	Zone-wise risk level
Citizen reports	Verification and prioritisation	Incident records and response tasks
Risk and occupancy status	Alert and resource coordination	Warnings, shelter status, dashboard

2. Objective and Expected Outcomes

The primary objective is to develop a working prototype that converts heterogeneous flood data into timely and actionable early-warning and evacuation information for Assam. The system is intended as decision support for authorities and as a communication platform for citizens; it is not a replacement for official emergency command.

- Collect and integrate IoT, weather, river, GIS, historical, and citizen-generated data.
- Predict flood risk for defined geographic zones using a hybrid rules-and-ML approach.
- Provide multi-channel warnings with severity, location, time, and recommended action.
- Display affected areas, shelters, shelter occupancy, available resources, and recommended routes.

- Enable citizens to submit geotagged emergency reports and authorities to verify and act on them.
- Demonstrate containerized deployment, API integration, database persistence, testing, and collaborative GitHub workflow.

Expected outcome	Success indicator
Early warning	Risk classification and alert generated within the prototype response target
Reliable prediction	Measured accuracy, precision, recall, and confusion matrix on test data
Evacuation support	Routes avoid flooded segments and reach available shelters
Operational dashboard	Authorities can view risk, incidents, shelters, and resources
Resilience	System handles missing readings and continues with last-known valid data
Maintainability	Services run through Docker Compose and code is organized by feature

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

ID	Requirement
FR-01	The system shall register sensors and receive water-level and rainfall readings.
FR-02	The system shall import weather, river, GIS, and historical data through APIs or files.
FR-03	The system shall validate, timestamp, store, and visualise incoming observations.
FR-04	The system shall calculate zone-wise flood risk and warning severity.
FR-05	The system shall issue alerts by dashboard, SMS/e-mail simulation, and push notification simulation.
FR-06	The system shall display affected areas and recommend evacuation routes.
FR-07	The system shall maintain shelter capacity, occupancy, resources, and status.
FR-08	The system shall allow citizens to submit emergency reports with category, description, and location.
FR-09	The system shall provide role-based dashboards for citizens, responders, authorities, and administrators.

FR-10	The system shall record audit logs for alerts, updates, verification, and administrative changes.

3.2 Non-Functional Requirements

Attribute	Requirement
Performance	Dashboard queries should return within 3 seconds under prototype load; risk processing should be near real time.
Availability	The system should continue operating when an individual sensor or external data source is unavailable.
Security	Use authentication, role-based authorisation, input validation, secure passwords/tokens, and audit logs.
Scalability	Services and database design should support additional districts, sensors, shelters, and users.
Usability	Alerts must be concise, readable, localised where possible, and actionable.
Maintainability	Use modular services, version control, documented APIs, tests, and Docker Compose.
Privacy	Collect only necessary personal data and restrict access to sensitive citizen and location records.

3.3 Constraints and Assumptions

Constraints include unreliable connectivity, sensor failures, incomplete ground truth, limited access to live institutional APIs, changing road conditions, language diversity, finite computing resources, and the safety implications of incorrect warnings. The prototype assumes that authorised agencies can supply or approve sensor locations, shelter data, road-network information, and evacuation policy. Model outputs are recommendations and must be verified by responsible officials before mass evacuation orders.

4. Application of Relevant Course Knowledge and Concepts

Course concept	Application in the assignment
Requirements engineering	Stakeholder analysis, functional/non-functional requirements, assumptions, acceptance criteria
Agile/Scrum	Product backlog, MoSCoW prioritisation, sprint goals, Scrum board, reviews and retrospectives
Software architecture	Service decomposition, REST APIs, event/data flow, containerised deployment
Database systems	Relational entities for users, sensors, observations, alerts, shelters, routes, and reports
AI/ML	Feature engineering, classification/regression, validation,

	precision/recall, drift awareness
GIS	Spatial layers, point-in-polygon analysis, hazard overlays, route and shelter mapping
IoT and networks	Sensor registration, MQTT/HTTP ingestion, timestamps, validation, retry and buffering
Software testing	Unit, integration, system, usability, performance, and security-oriented tests
DevOps and collaboration	Git branches, pull requests, code reviews, Docker Compose, environment configuration
Project management	Risk register, deliverables, sprint planning, definition of done, traceability

5. Design, Proposed Solution and Methodology

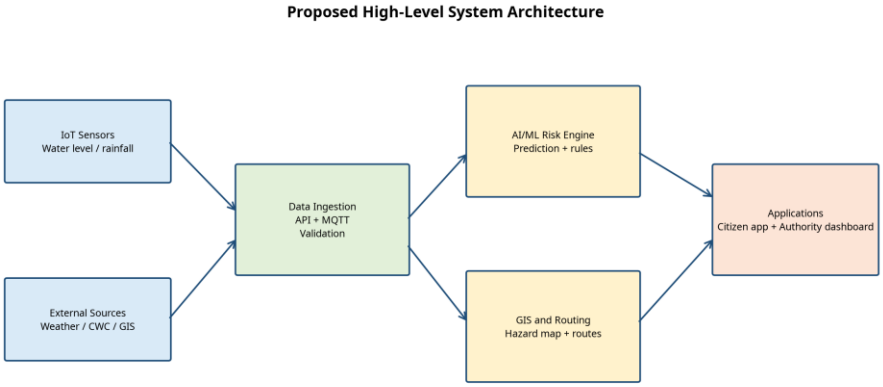

Figure 1. Proposed system architecture.

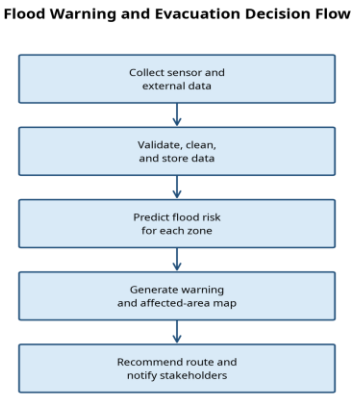

Figure 2. Flood-warning and evacuation decision flow.

5.1 Main Components

Component	Responsibility
Data ingestion service	Receives MQTT/HTTP readings and external API data; validates schema and timestamps.
Data and database layer	Stores users, readings, predictions, alerts, reports, shelters, resources, and audit events.
Risk engine	Combines thresholds and ML predictions; produces risk score, severity, confidence, and explanation.
GIS service	Creates hazard layers, identifies affected zones, checks road segments, and computes routes.
Alert service	Creates targeted alert messages and tracks delivery status and acknowledgement.
Shelter/resource module	Maintains capacity, occupancy, medical needs, food, water, boats, and other resources.
Citizen interface	Shows alerts and maps and accepts emergency reports.
Authority dashboard	Supports monitoring, verification, response coordination, and analytics.

5.2 Data Model

Entity	Important fields
User	user_id, role, name, phone/e-mail, location, consent
Sensor	sensor_id, type, latitude, longitude, zone_id, status
Observation	observation_id, sensor_id, timestamp, value, quality_flag
RiskPrediction	zone_id, timestamp, score, severity, confidence, model_version
Alert	alert_id, severity, zone, message, created_at, channel, delivery_status
Shelter	shelter_id, name, location, capacity, occupancy, status
EmergencyReport	report_id, citizen_id, location, category, description, priority, verification_status
Resource	resource_id, shelter_id, item, quantity, last_updated

6. Agile Scrum Plan and Product Backlog

The project follows Scrum. The product backlog is prioritised using MoSCoW: Must Have features are essential for a usable early-warning prototype; Should Have features

improve operational value; Could Have features are desirable enhancements; and Won't Have features are excluded from the current release.

Priority	Backlog item	Acceptance criterion	Sprint
Must	Sensor and API data ingestion	Valid readings are stored with timestamp and quality flag	1
Must	Risk scoring and severity classification	A zone receives a score and warning level from test inputs	2
Must	Authority dashboard	Authority can view current risk and active alerts	2
Must	Citizen alerts	Citizen receives location and severity-specific message	3
Must	GIS hazard map and route	Map shows affected area and route to available shelter	3
Must	Shelter occupancy	Capacity and current occupancy are updated and displayed	3
Should	Citizen emergency reporting	Report is created, prioritised, and visible to authority	4
Should	Resource monitoring	Authority can update and inspect shelter resources	4
Should	Audit logs and analytics	Actions and key metrics are recorded and summarised	4
Could	Multilingual message templates	Alert templates are available in selected local languages	5
Could	Offline-first mobile mode	Recent alerts remain available during temporary loss of connectivity	5
Won't	Autonomous evacuation ordering	Final orders remain a human authority function in this release	—

Sprint	Goal	Key deliverables
Sprint 1	Foundation and data layer	Repository, Docker Compose, database schema, sensor simulator, ingestion API
Sprint 2	Risk intelligence and dashboard	Risk engine, prediction endpoint, authority dashboard, initial tests
Sprint 3	GIS, alerts and shelters	Hazard map, routing, alert service, shelter module, citizen alert view
Sprint 4	Reports, resources and validation	Citizen reports, resource tracking, integration tests, performance tests
Sprint 5	Hardening and demonstration	Security checks, documentation, screenshots, review, retrospective

Scrum board columns: Product Backlog → To Do → In Progress → Code Review → Testing → Done. A work item is Done only when its code is committed to a feature branch, reviewed through a pull request, tested, documented, and demonstrated against its acceptance criterion.

7. Use-Case Analysis and User Stories

Actor	Primary use cases
Citizen	View warnings, receive alerts, view shelter and route, submit emergency report
Disaster-management authority	Monitor zones, verify reports, issue/approve alerts, assign response actions
Emergency response agency	View incidents, accept tasks, update rescue/resource status
Administrator	Manage users, sensors, zones, shelters, thresholds, audit logs
System/External service	Send sensor data, provide weather/river data, deliver notifications, provide map data

7.1 User Stories

- As a citizen living near a river, I want to receive an early warning for my location so that I can prepare or evacuate before conditions become dangerous.
- As an authority officer, I want to view risk levels on a map so that I can identify priority zones and coordinate response.
- As a responder, I want to see verified emergency reports and recommended routes so that rescue teams can reach affected people safely.
- As a shelter manager, I want to update occupancy and resources so that authorities do not direct people to unavailable shelters.
- As an administrator, I want to configure sensors, users, thresholds, and roles so that the platform remains reliable and secure.

8. Algorithm, Pseudocode and Flowchart

A hybrid approach is proposed. Rule-based thresholds provide transparent safety safeguards, while an ML model can learn relationships among rainfall, water level, trend, elevation, and historical outcomes. If the ML service is unavailable, the threshold engine continues operating.

```
ALGORITHM GenerateFloodWarning(zone, current_data):
    data <- validate_and_normalise(current_data)
    IF data is invalid: return last_valid_result with LOW_CONFIDENCE
    trend <- calculate_water_level_trend(data.water_levels)
    features <- [water_level, rainfall_1h, rainfall_24h, trend, elevation, vulnerability]
    ml_score <- model.predict(features)
    rule_score <- threshold_score(water_level, rainfall_24h, trend)
    risk_score <- weighted_average(ml_score, rule_score)
    severity <- classify(risk_score)
    IF severity in {WARNING, SEVERE_WARNING}:
        affected_area <- GIS.identify_affected_area(zone, risk_score)
        shelters <- find_available_shelters(affected_area)
        routes <- GIS.recommend_safe_routes(affected_area, shelters)
        alert <- create_location_specific_alert(zone, severity, routes)
        send_alert(alert, citizens, authorities, responders)
    store_prediction(zone, risk_score, severity, confidence)
    return risk_score, severity, routes
```

9. Implementation, Source Code and Environment / Tools Used

The implementation is organised as a modular full-stack prototype. The frontend presents dashboards and maps; the backend exposes REST endpoints; the database persists operational data; the risk service calculates predictions; and Docker Compose runs the components consistently across development environments.

Layer	Proposed technology
Frontend	React or HTML/CSS/JavaScript with a map component
Backend API	Python FastAPI or Node.js Express
Database	PostgreSQL with spatial extensions where available
AI/ML	Python, pandas, scikit-learn; hybrid rules plus classifier
GIS and maps	Leaflet/OpenStreetMap-compatible layers; routing service or simulated network
Messaging	REST notification adapter; SMS/e-mail/push simulation for prototype
Containerisation	Docker and Docker Compose
Collaboration	Git, GitHub feature branches, pull requests, reviews, issue tracking
Testing	Pytest/Jest, API integration tests, browser testing, load-test scripts

9.1 Representative Backend Source Code

```
@app.post("/api/risk/predict")
def predict_risk(request: RiskRequest):
    validate_zone(request.zone_id)
    trend = request.water_level - request.previous_water_level
    rule_score = threshold_score(request.water_level, request.rainfall_24h, trend)
    ml_score = risk_model.predict([[request.water_level, request.rainfall_24h,
                                    trend, request.elevation]])[0]
    score = 0.5 * rule_score + 0.5 * ml_score
    severity = classify_score(score)
    result = save_prediction(request.zone_id, score, severity)
    if severity in ["WARNING", "SEVERE_WARNING"]:
        create_and_queue_alert(request.zone_id, severity)
    return result
```

9.2 Container Services

```
services:
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
  api:
    build: ./backend
    ports: ["8000:8000"]
    depends_on: [db]
  risk-engine:
    build: ./risk-engine
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: disaster_db
      POSTGRES_USER: app_user
      POSTGRES_PASSWORD: change_in_development_only
```

10. Test Cases and Expected/Actual Results

This section presents evidence from the implemented Assam Flood Early Warning & Evacuation application. The screenshots demonstrate the working user interface, live-warning banner, authority alert-broadcast module, severity classification, location-specific instructions, and shelter/helpline information. The displayed values are marked

as demonstration data where appropriate and should not be treated as an official operational forecast.

10.1 Main Application and Navigation

The main application provides a unified entry point for flood monitoring, AI prediction, GIS mapping, alerts, evacuation, shelter management, and resource monitoring. The top navigation shows the major modules available to users. A prominent live-warning banner is used to communicate the current warning state and identify affected locations.

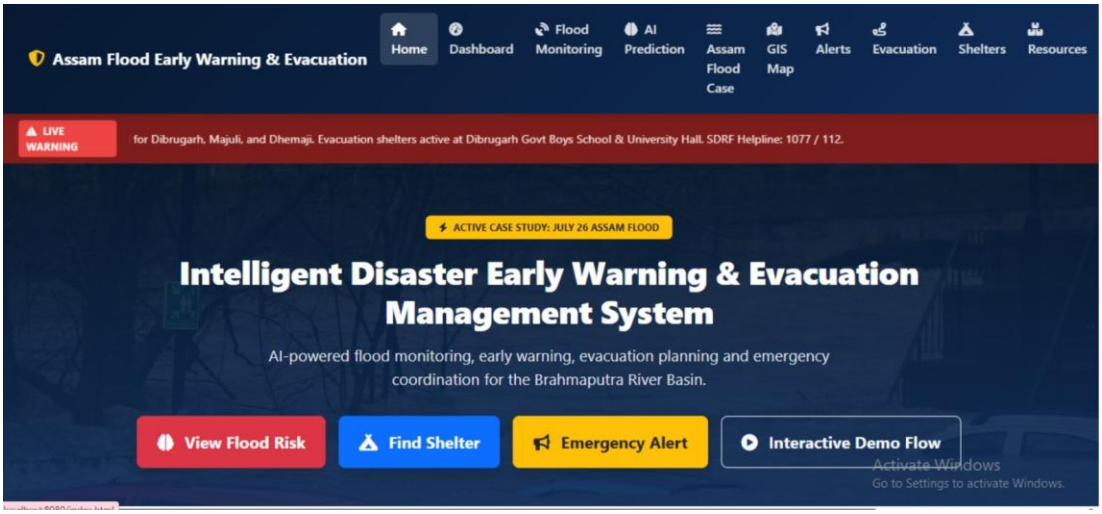

Figure 3. Implemented Assam Flood Early Warning and Evacuation Management home screen.

The home screen includes the actions View Flood Risk, Find Shelter, Emergency Alert, and Interactive Demo Flow. It also displays the active case-study context and links to the operational modules required by the assignment.

10.2 Emergency Alert Broadcast Centre

The Emergency Alerts Broadcast Centre supports authority-controlled publication of official alerts. It presents alert severity, affected districts or localities, explanatory text, public instructions, issuing authority, and active status. This directly demonstrates the requirement for location-specific, multi-level early-warning communication.

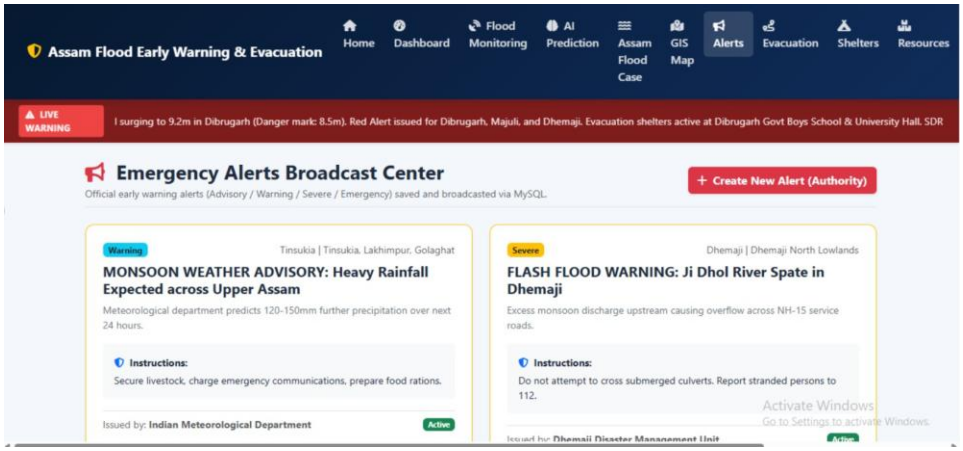

Figure 4. Implemented Emergency Alerts Broadcast Centre showing warning and severe alerts.

The screenshot demonstrates a Warning alert for heavy rainfall across Upper Assam and a Severe alert for flash-flood risk in Dhemaji. The interface also includes the Create New Alert (Authority) control, which supports authorised alert creation and broadcasting.

10.3 Critical Warning and Evacuation Instructions

The alert cards provide actionable instructions rather than displaying risk information alone. The implementation includes a severe embankment-breach threat in Majuli and an emergency Brahmaputra-spilling warning in Dibrugarh. The instructions direct residents to shelters or higher ground, advise the use of emergency kits and waterproof document storage, identify unsafe roads, and provide the SDRF helpline.

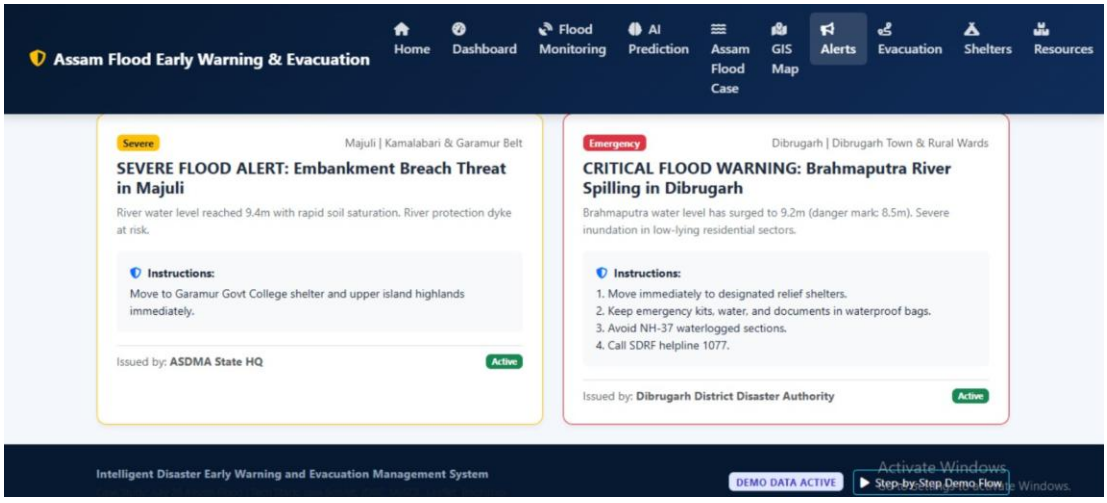

Figure 5. Implemented severe and emergency flood alerts with location-specific instructions.

The demonstrated output confirms that the system distinguishes between Severe and Emergency levels, shows the issuing organisation, marks alerts as Active, and provides response instructions for the affected area. These functions support the citizen, authority, and emergency-response perspectives defined in the requirements.

10.4 Implementation Evidence Summary

Evidence ID	Observed implementation output	Requirement demonstrated
E-01	Unified navigation for monitoring, AI prediction, GIS map, alerts, evacuation, shelters, and resources	Integrated full-stack disaster-management interface
E-02	Live warning banner for Dibrugarh, Majuli, and Dhemaji with shelter and helpline information	Location-specific warning communication
E-03	Authority alert broadcast centre with Warning and Severe alert cards	Authority-controlled multi-level alert management
E-04	Dibrugarh river level of 9.2 m against an 8.5 m danger mark shown in the alert banner	Risk information and warning context
E-05	Majuli embankment-breach alert and Dibrugarh emergency alert with instructions	Actionable evacuation and preparedness guidance
E-06	Active status and issuing authority displayed on alert cards	Traceability and operational status

10.5 Test Cases and Actual Results

The following test cases are aligned with the features visible in the implementation screenshots. The actual results below describe the observed demonstration outputs; backend performance and model metrics should be added after executing the corresponding automated tests.

ID	Test case	Expected result	Actual result
TC-01	Open the application home page	Home screen loads with module navigation and warning area	Pass – home screen and navigation displayed
TC-02	Check the live-warning banner	Banner identifies warning severity, affected areas, shelters, and helpline	Pass – live warning banner displayed for Dibrugarh, Majuli, and Dhemaji
TC-03	Open Emergency Alerts	Alert broadcast centre lists official alert cards	Pass – broadcast centre displayed
TC-04	Inspect severity classification	Alerts are visually classified as Warning, Severe, or Emergency	Pass – Warning, Severe, and Emergency labels visible
TC-05	Inspect alert location and instructions	Each alert shows affected location and actionable guidance	Pass – district/locality and instructions displayed
TC-06	Verify authority traceability	Alert shows issuing authority and active status	Pass – issuer and Active status visible
TC-07	Verify shelter guidance	Warning interface identifies active shelters or higher-ground guidance	Pass – shelter and higher-ground instructions displayed
TC-08	Verify danger-mark context	Alert communicates measured level and danger mark where applicable	Pass – Dibrugarh 9.2 m and 8.5 m danger mark shown

Note: The screenshots demonstrate the implemented interface and demo data. Claims about live sensor accuracy, AI prediction performance, notification delivery time, and route safety require separate measured experiments and should be reported in Section 12 after execution.

11. Execution Screenshots and Outputs

Insert final screenshots in this section after implementation and execution. The required evidence should show the running Docker Compose services, database/API response,

authority dashboard, GIS hazard map, citizen alert screen, shelter occupancy view, and emergency-report workflow.

Screenshot ID	Required evidence	Caption to use
S-01	Docker Compose terminal and service list	Containerised services running successfully
S-02	API response for risk prediction	Zone-wise risk score and warning severity
S-03	Authority dashboard	Operational view of alerts, zones, and incidents
S-04	GIS map	Affected area, shelters, and recommended evacuation route
S-05	Citizen interface	Location-specific warning and recommended action
S-06	Shelter/resource screen	Occupancy and essential-resource monitoring
S-07	GitHub pull request	Feature branch review and collaborative development evidence

12. Results and Validation

Validation will compare system outputs against labelled test scenarios and, where available, historical or simulated flood observations. Since the assignment prototype may not have access to a continuous live sensor stream, simulation data should be clearly labelled. No simulated result should be presented as an official forecast.

Metric	Definition	Target for prototype
Accuracy	Correct risk classifications / all classifications	≥ 85% on held-out test scenarios
Precision	Correct high-risk predictions / all high-risk predictions	≥ 80%
Recall	Correct high-risk predictions / all actual high-risk cases	≥ 85%
F1-score	Harmonic mean of precision and recall	≥ 0.82
Alert latency	Time from accepted observation to alert queue	≤ 10 seconds in local test
API response time	95th percentile response for dashboard/risk endpoint	≤ 3 seconds
Delivery success	Successful notifications / queued notifications	≥ 95% in simulation

Route validity	Recommended routes not intersecting blocked segments	100% in controlled test network

A confusion matrix, latency table, alert-delivery log, and screenshots should be attached to the final submission as evidence. Validation must also include failure scenarios such as missing readings, stale data, duplicate messages, unavailable shelters, invalid coordinates, and loss of the external weather service.

13. Analysis, Comparison, Trade-offs and Justification

Approach	Advantages	Limitations	Decision
Rules only	Transparent, easy to audit, works with little data	May not capture complex patterns; thresholds need calibration	Use as safety layer
ML only	Can model non-linear patterns and improve with data	Needs labelled data; may be difficult to explain; can fail under drift	Use as supporting predictor
Hybrid rules + ML	Combines transparency, fallback operation, and predictive capability	More components and calibration effort	Selected final approach
Centralised monolith	Simple initial deployment	Harder to scale and isolate failures	Not selected for final architecture
Containerised modular services	Independent development, reproducible deployment, scalable boundaries	Operational complexity and networking overhead	Selected for assignment
Single-channel alerts	Simple and inexpensive	High risk of missed warnings	Not selected
Multi-channel alerts	Improves reach and redundancy	Requires delivery adapters and monitoring	Selected for final design

The hybrid architecture is justified because disaster warnings require both predictive capability and explainable safeguards. A purely data-driven model may be unreliable when historical labels are limited or environmental conditions change. A purely rule-

based system is transparent but less adaptable. The selected design also separates prediction from final authority action, reducing the risk of treating an automated score as an unquestionable evacuation order.

14. Broader Considerations and SDG Relevance

The project supports disaster-risk reduction by improving access to timely information, strengthening coordination, and helping communities prepare for and respond to floods. It is directly relevant to **United Nations Sustainable Development Goal 11: Sustainable Cities and Communities**, particularly the objective of making human settlements safer and more resilient to disasters [1]. It also relates to SDG 13 through climate adaptation and resilience planning [2].

- Accessibility: Use readable interfaces, audio or SMS alternatives, clear icons, and local-language message templates where feasible.
- Equity: Give attention to remote villages, older adults, persons with disabilities, low-connectivity users, and people without smartphones.
- Privacy: Minimise personal data, obtain consent, restrict sensitive location access, and define retention rules.
- Trust and accountability: Show alert source, timestamp, severity, confidence, and recommended action; maintain human review for official orders.
- Environmental responsibility: Use efficient cloud resources, durable sensor deployment, and maintainable software.
- Responsible AI: Monitor false alarms, missed events, data bias, model drift, and explainability of high-risk predictions.

15. Individual Contribution of Group Members

Member	Register number	Proposed contribution
K R Gayana	192471039	Project coordination, problem formulation, Scrum planning, and final integration
S Keerthana	192471047	Requirements analysis, use cases, user stories, and documentation
N Ramya	192511403	Frontend interface, citizen alert view, and usability testing
A Saruhaasini	192511400	Backend APIs, database schema,

		authentication, and audit logging
S Nithiyasri	192572082	AI/ML risk prediction, data preparation, and validation metrics
M Hashetha	192521162	GIS mapping, evacuation routing, Docker deployment, and system testing

The contribution allocation is a proposed assignment of responsibilities and should be updated to reflect the actual work completed by each member. All members should participate in code reviews, sprint reviews, testing, and the final presentation.

16. One-Page Individual Reflections

K R Gayana

I contributed to coordinating the assignment and converting the broad disaster-management problem into a feasible product backlog. A key design decision was to develop the system incrementally using Scrum, because the project includes several interconnected components and requirements may change after demonstrations. The main challenge was balancing the ambition of real-time AI, GIS, IoT, and cloud integration with the time and data available for a prototype. I learned how project planning, prioritisation, traceability, and integration influence the quality of a technical solution. The system contributes to SDG 11 by supporting safer and more resilient communities. This assignment helped me develop the mapped course outcomes related to requirements analysis, system design, teamwork, and project management.

S Keerthana

My contribution focused on requirements, stakeholders, use cases, and user stories. I learned that citizens, authorities, responders, and administrators need different information and permissions, so a single generic interface would not be suitable. The major challenge was writing requirements that were specific enough to test without assuming unrealistic access to live government data. MoSCoW prioritisation helped us separate essential warning functions from future enhancements. The project relates to SDG 11 because inclusive access to disaster information can reduce vulnerability. The assignment strengthened my ability to analyse user needs, define acceptance criteria, document constraints, and connect requirements to design decisions.

N Ramya

I worked on the proposed frontend and citizen-facing interactions. The main design decision was to keep warnings concise and action-oriented, showing severity, affected location, time, shelter information, and recommended next steps. A challenge was designing for users with limited connectivity and different levels of digital literacy. I learned that usability and accessibility are safety requirements in an emergency system, not merely visual improvements. The system supports resilient communities under SDG 11 by making warnings and evacuation information easier to understand. This assignment improved my knowledge of interface design, API integration, user testing, and translating technical data into meaningful information.

A Saruhaasini

My contribution covered backend APIs, data persistence, authentication, and auditability. I designed the backend around separate entities for sensors, observations, predictions, alerts, shelters, reports, and users so that the data could be traced through the warning process. The main challenge was handling invalid, duplicate, missing, or delayed readings without generating misleading results. I learned the importance of validation, role-based access, logging, and modular service design. The project contributes to SDG 11 by strengthening digital infrastructure for disaster response. It helped me achieve learning outcomes related to database design, API development, security, and software integration.

S Nithiyasri

I contributed to the AI/ML risk-prediction component and validation metrics. The selected hybrid approach combines transparent thresholds with a predictive model, which is appropriate because safety decisions need both adaptability and explainability. The greatest challenge was recognising that a model cannot be trusted merely because it produces a numerical score; it must be evaluated using precision, recall, false alarms, missed events, and confidence. I learned about feature engineering, data quality, model validation, fallback logic, and responsible AI. The system supports SDG 11 and climate resilience by improving preparedness. This assignment developed my skills in machine learning application, experimental validation, and interpreting results responsibly.

M Hashetha

My contribution focused on GIS mapping, evacuation-route recommendations, containerisation, and testing. The design decision was to treat route safety and shelter availability as dynamic constraints rather than simply displaying a static map. The main

challenge was representing changing road conditions and incomplete spatial data in a prototype. Docker Compose helped create a repeatable environment for the frontend, backend, risk engine, and database. I learned how spatial data, deployment, testing, and system reliability interact in a real application. The project is relevant to SDG 11 because safer evacuation planning improves community resilience. The assignment strengthened my skills in GIS concepts, DevOps, integration testing, and deployment documentation.

17. Conclusion, Limitations and Possible Improvements

This assignment proposes an intelligent, containerized, full-stack platform for Assam flood early warning and evacuation management. By integrating sensor observations, weather and river information, GIS, AI/ML, shelter data, citizen reports, and authority workflows, the system can provide more timely and actionable support than disconnected information channels. The proposed Scrum plan, modular architecture, algorithms, tests, and validation metrics provide a practical path from problem definition to demonstrable prototype.

17.1 Limitations

- A prototype may rely on simulated or limited datasets rather than a continuous live sensor network.
- Prediction quality depends on the availability, accuracy, representativeness, and freshness of labelled data.
- Road conditions, shelter capacity, and communication coverage can change faster than the database is updated.
- External APIs and map services may impose rate limits, availability constraints, licensing conditions, or cost.
- The prototype cannot guarantee that every citizen receives or follows an alert.
- Evacuation recommendations require official verification and must not replace trained emergency decision-makers.

17.2 Possible Improvements

- Integrate approved live feeds from river gauges, weather services, and disaster-management authorities.
- Train and recalibrate models with district-level historical flood and rainfall data.
- Add multilingual, voice, USSD, SMS, and offline-first communication channels.
- Use satellite imagery and change detection to improve affected-area mapping.
- Add sensor health monitoring, edge buffering, digital signatures, and tamper detection.
- Introduce role-based workflow approvals, incident command features, and post-event analytics.
- Conduct field pilots with communities and authorities and measure real warning lead time and user comprehension.

18. References

[1] United Nations Department of Economic and Social Affairs. “Goal 11: Sustainable Cities and Communities.” Available at: https://sdgs.un.org/goals/goal11

[2] United Nations Department of Economic and Social Affairs. “Disaster Risk Reduction.” Available at: https://sdgs.un.org/topics/disaster-risk-reduction

[3] Government of Assam, Water Resources Department. “Flood Management.” Available at: https://waterresources.assam.gov.in/portlets/flood-management

[4] Central Water Commission, Government of India. “Flood Forecasting / Hydrological Observation.” Available at: https://www.cwc.gov.in/flood-forecasting-hydrological-observation

[5] Central Water Commission, Government of India. “Flood Forecast.” Available at: https://ffs.india-water.gov.in/

[6] Docker Documentation. “Docker Compose.” Available at: https://docs.docker.com/compose/

[7] GitHub Documentation. “About pull requests.” Available at: https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/about-pull-requests

[8] OpenStreetMap Foundation. “OpenStreetMap Copyright and License.” Available at: https://www.openstreetmap.org/copyright

[9] NIST. “Artificial Intelligence Risk Management Framework.” Available at: https://www.nist.gov/itl/ai-risk-management-framework